

Evaluation and Anomaly Detection

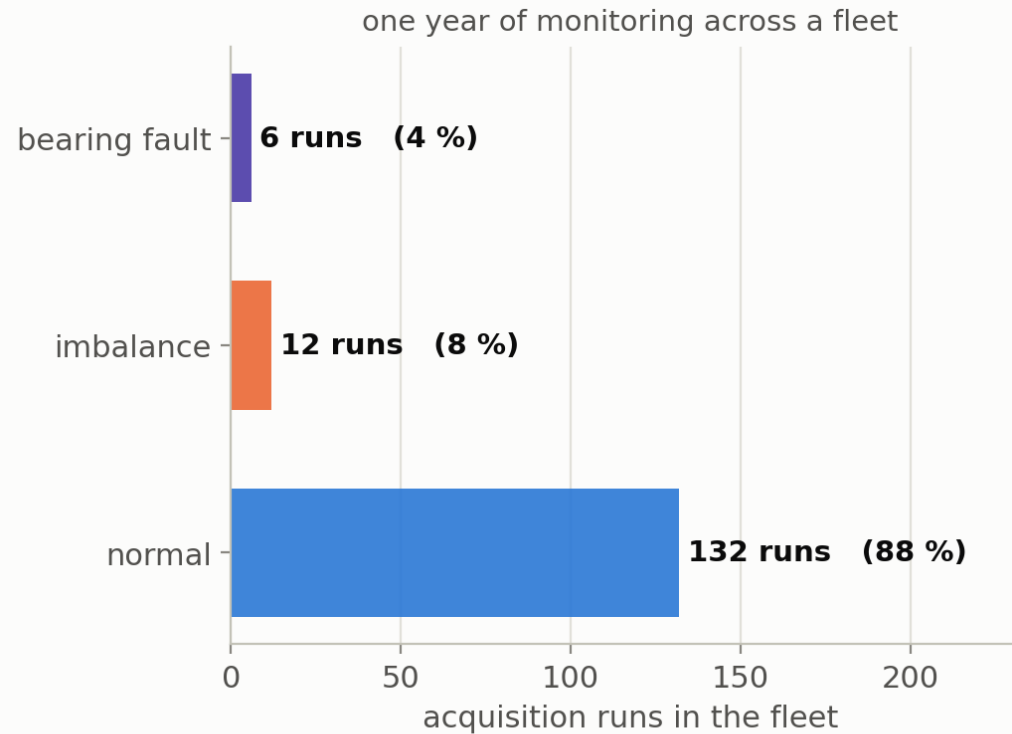
What accuracy hides — and what you can still build with no fault labels at all

- A fleet with industrial prevalence: 150 acquisitions, six bearing spalls
- The confusion matrix, precision, recall — and what accuracy was averaging over
- Cost asymmetry: turning a stated cost ratio into a threshold
- Cross-validation when the positive class is six runs
- Detectors trained on healthy data alone, and how to set their threshold
- Concept drift: why every threshold has a shelf life

Notebook: `iaiiot26_Lect10_nb.ipynb`

The 0.880 that means nothing

- A classifier that always predicts 'normal' needs no features, no training and no data — and on this fleet it scores exactly what Lecture 9's best model did.



"always normal" — accuracy 0.880

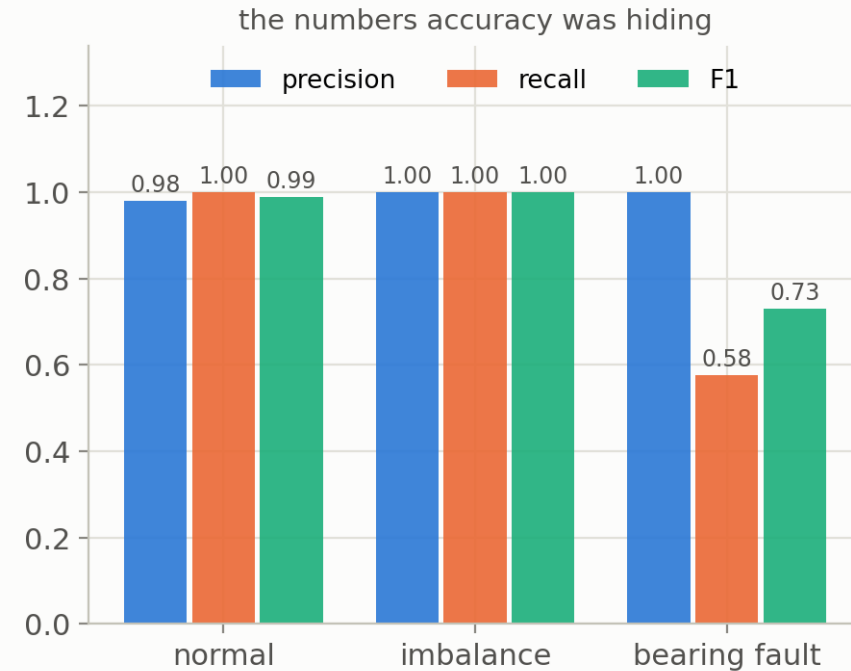
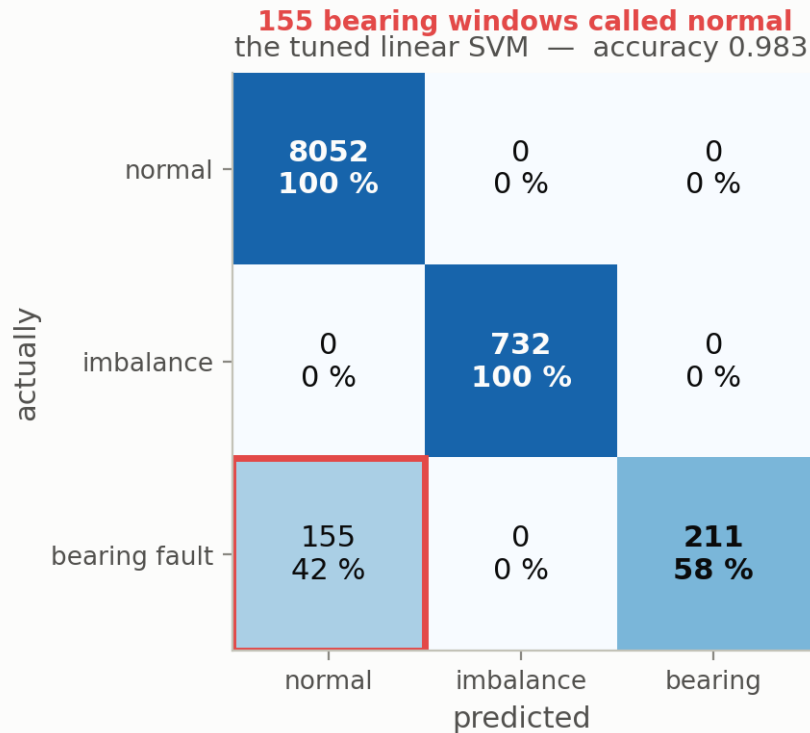
actually	normal	8052	0	0
	imbalance	732	0	0
	bearing fault	366	0	0
		normal	imbalance	bearing
		predicted		

A classifier that has never predicted a fault scores 0.880 — the same accuracy Lecture 9's best model earned on balanced data.

6 bearing spalls in 150 acquisitions. Always compute the majority-class accuracy before you report anything — if you cannot beat 0.880 by a defensible margin, you have built nothing.

What accuracy was averaging over

- Lecture 9's linear SVM, unchanged, on the fleet. Every window predicted out of fold by a model that never saw its run.

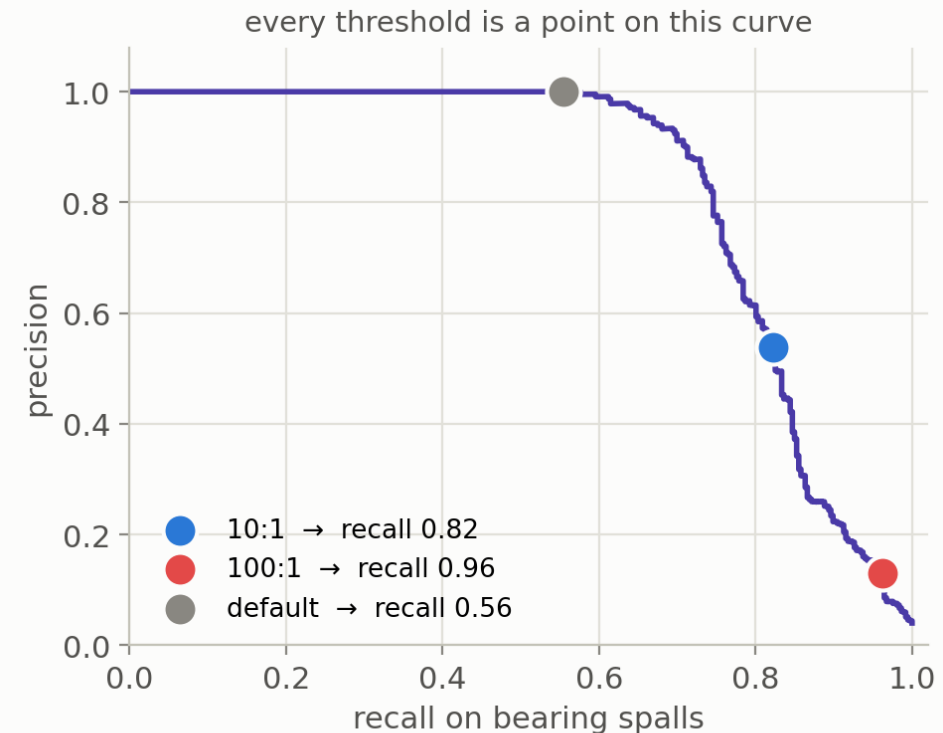
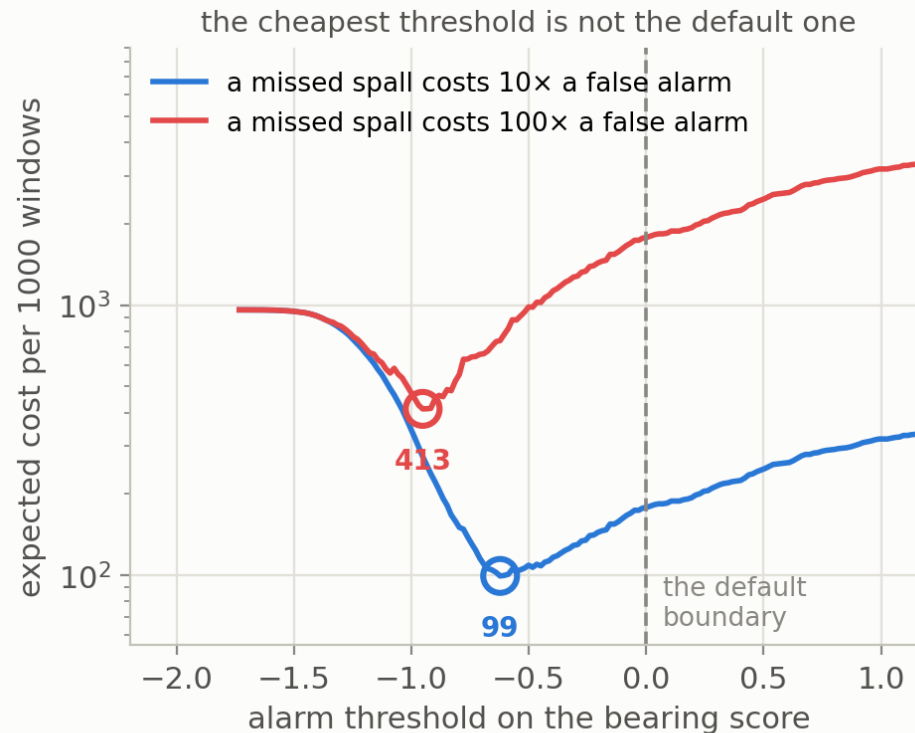


Accuracy 0.983 looks excellent. It is the average of a class we always get right and a class we miss 42 % of the time.

Accuracy 0.983. Bearing recall 0.577 — more than four spalls in ten go past unnoticed — and bearing precision 1.000: when it does fire, it is never wrong.

A missed spall is not a wasted inspection

- The default decision boundary picks a point on the precision–recall trade for reasons that have nothing to do with your plant. Choose it deliberately instead.

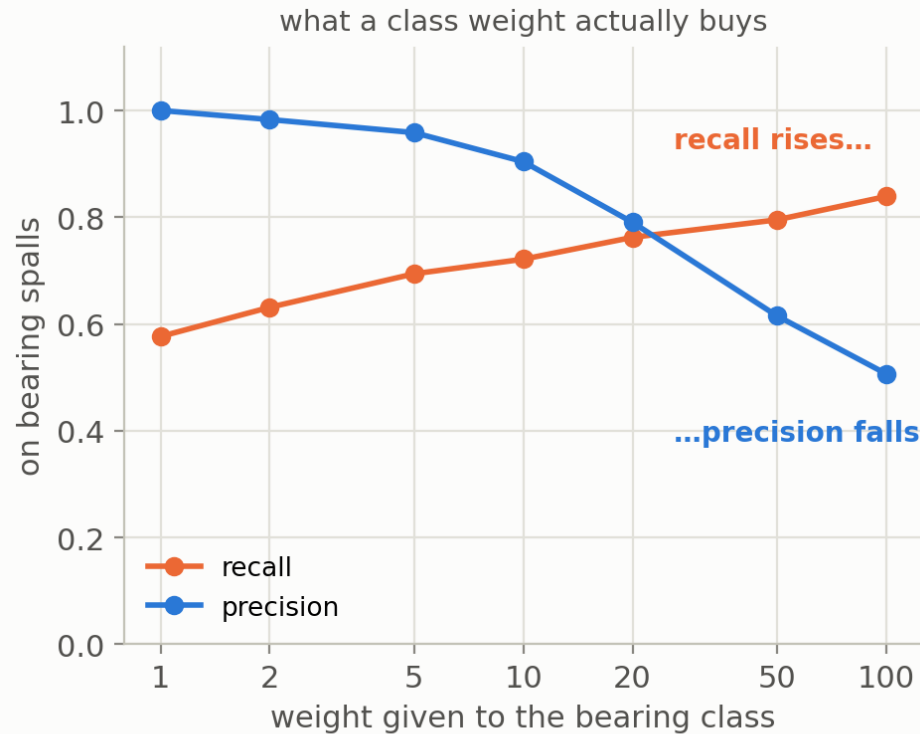


The model did not change. Only the threshold moved — and at 100:1 that is worth 1368.8 cost units per 1000 windows.

At 10:1 the cheapest threshold catches 82 % of spalls for 2.9 % false alarms. The model never changed — only the number its score is compared against.

Class weights move you along the curve

- Telling the model the rare class matters more does change its behaviour — but it does not give it any new information about bearing spalls.



class_weight='balanced'

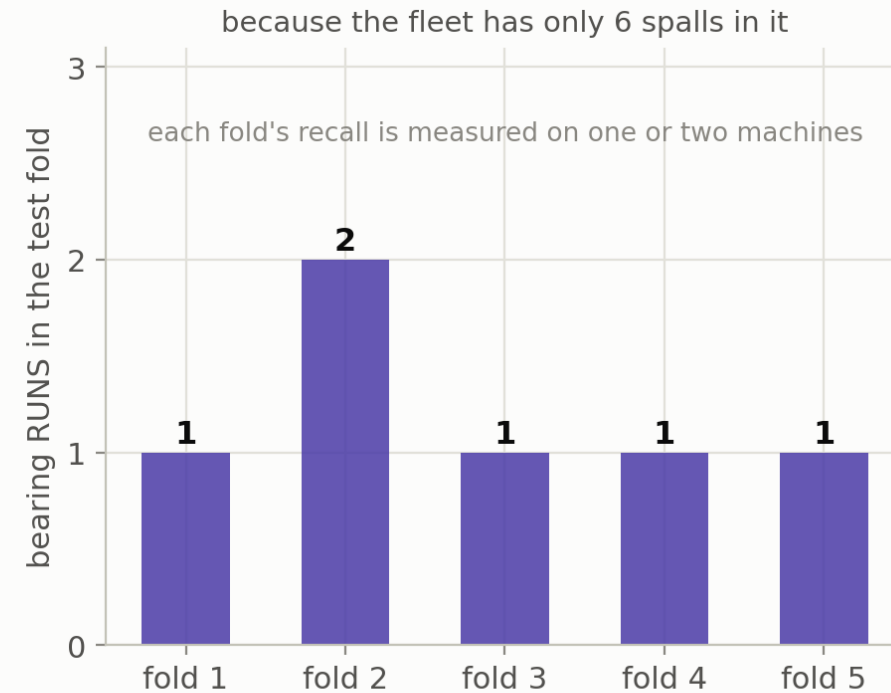
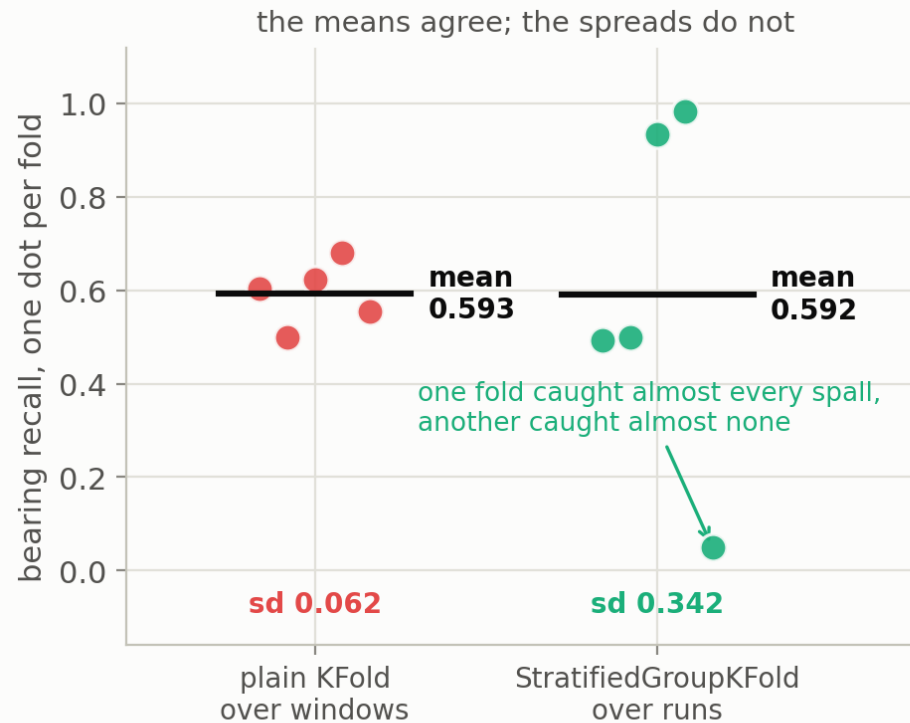
	normal	imbalance	bearing
normal	8002 99 %	0 0 %	50 1 %
imbalance	0 0 %	730 100 %	2 0 %
bearing fault	95 26 %	0 0 %	271 74 %
	normal	imbalance	bearing
	predicted		

Bearing recall 0.577 → 0.740, precision 1.000 → 0.839. Weights move you along the curve; they do not create information.

Recall 0.577 → 0.740, precision 1.000 → 0.839, and overall accuracy barely moves (0.983 → 0.984) — one more reason not to report it.

Five folds, five different answers

- Grouping by run was Lecture 8's rule. With a rare class there is a second problem it does not solve: there may only be one positive run per fold.



Plain KFold gives a comfortable, tight, meaningless spread. Grouping tells you the truth: this recall is worth 0.59 ± 0.34 .

The means agree (0.593 against 0.592); the spreads do not (sd 0.062 against 0.342). With 6 spalls in the fleet, this recall is worth about ± 0.3 .

Now take the labels away

- Everything so far assumed a column saying which runs were faulty. Somebody had to write that column, machine by machine, after the fact.

what a plant actually has after a year (proportions illustrative)

thousands of hours of healthy running

nobody labelled it — the machine simply did not break

known failures

a handful, recorded after the fact

everything in between

running rough, running hot, nobody wrote it down

supervised learning needs

labelled examples of every fault you want to catch — which is the one part nobody has

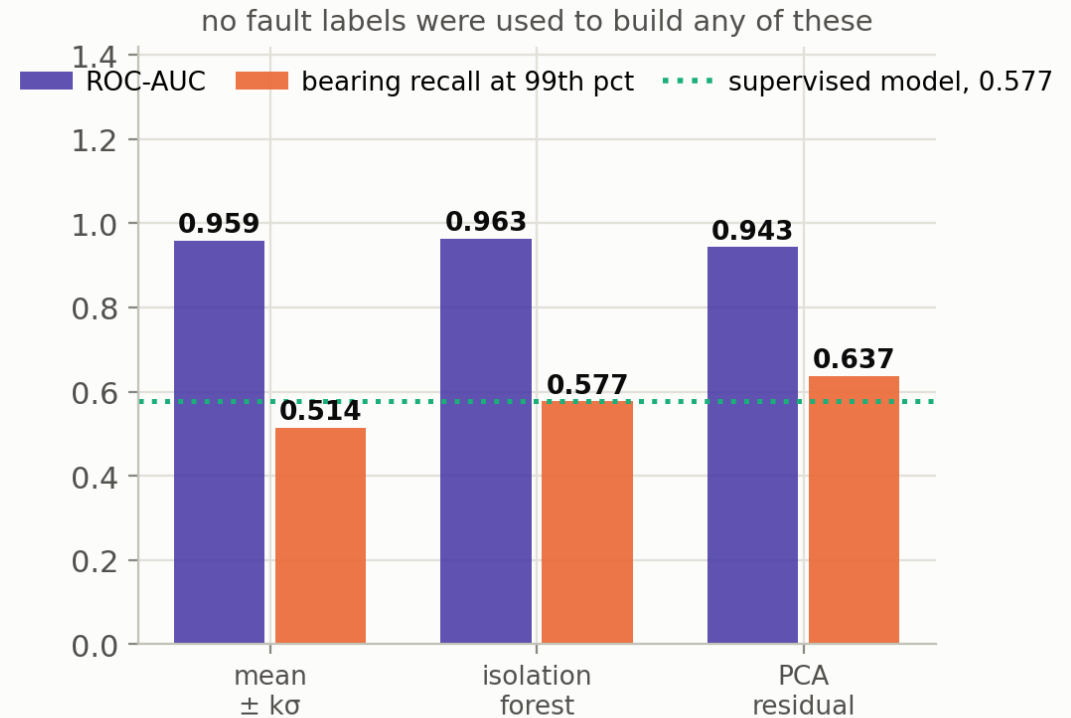
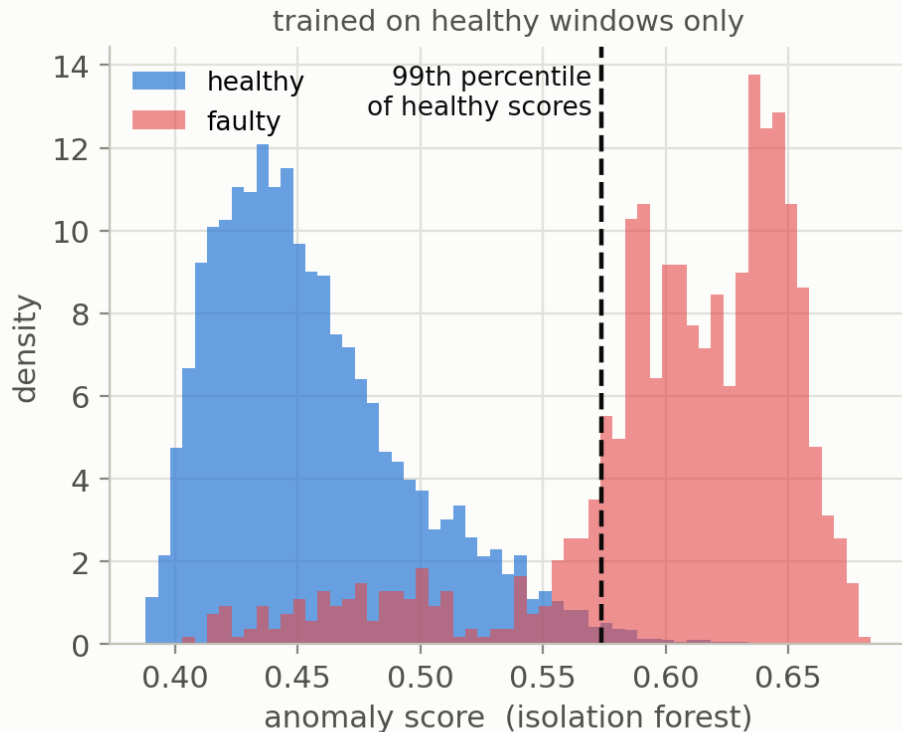
anomaly detection needs

only the healthy data — and asks a different question: “is this machine behaving as it always has?”

A supervised model is blind to a failure mode it has never seen; a detector notices it but cannot name it. That trade is why almost every real monitoring system starts as a detector.

Three detectors, no fault labels

- All three fitted on healthy windows only. The fault windows exist here purely so we can measure how they did — a real deployment would not have them.

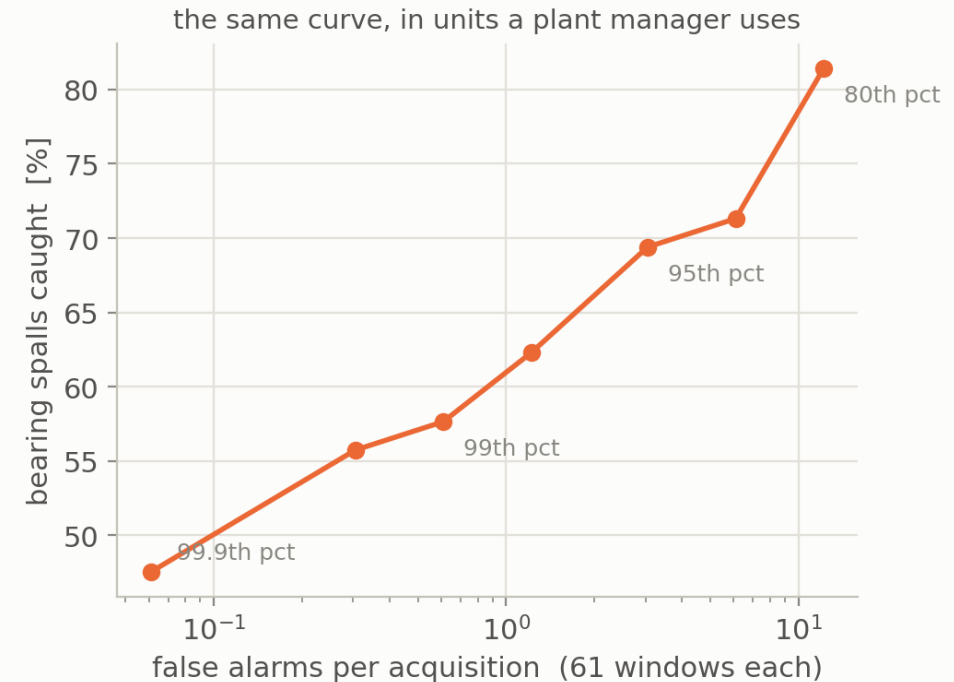
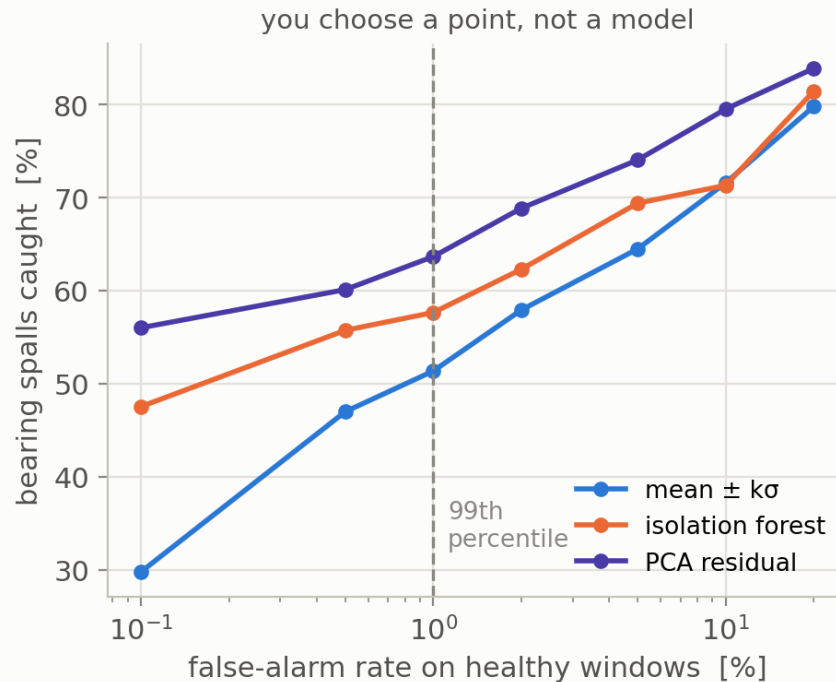


The PCA residual misses imbalance almost entirely (0.56 against 0.95) — a fault that lives inside the healthy subspace reconstructs perfectly.

The isolation forest reaches bearing recall 0.577 having never seen a fault — the same as the supervised model, which was given every label. It pays 1.0 % false alarms for it.

The threshold is a business decision

- With no fault labels you cannot choose the threshold by maximising anything. You state a tolerable false-alarm rate and read it off the healthy score distribution.

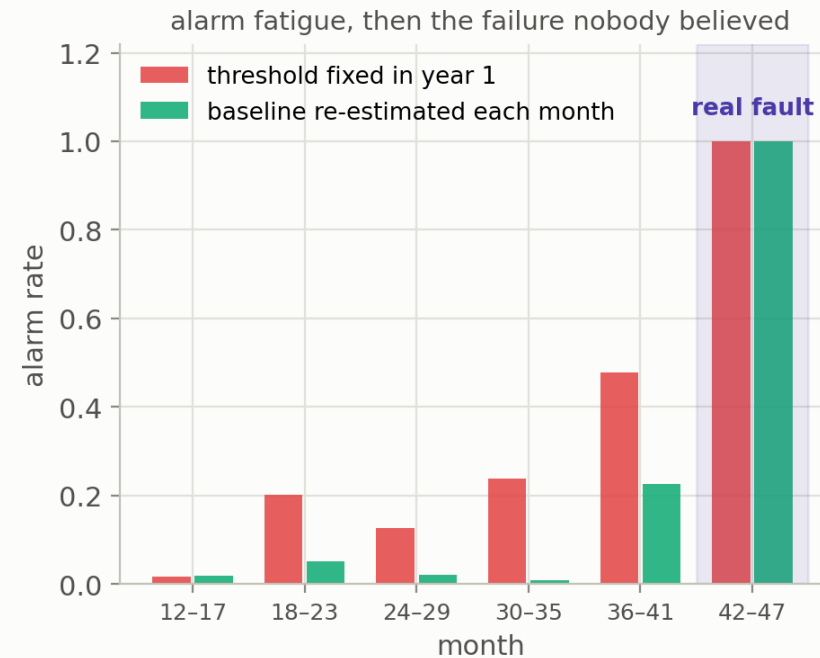
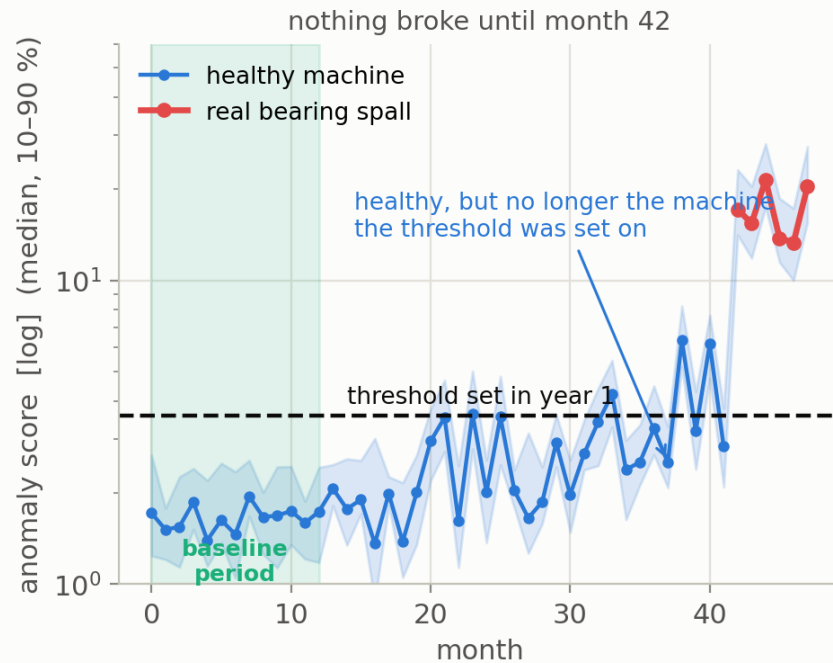


The threshold is not a hyperparameter to tune. It is a statement about how many false alarms your maintenance team will tolerate before they stop answering.

At the 99th percentile of healthy scores, about 1.0 % of healthy windows alarm — roughly 0.6 per acquisition — and 58 % of spall windows are caught.

A threshold has a shelf life

- One healthy machine, measured monthly for four years. Mounts relax, ambient noise rises, a bearing is replaced with a different brand. Nothing breaks until month 42.



By year four the fixed threshold is firing on 48 % of windows from a healthy machine; the re-estimated one on 7 % across the whole period. Both catch the real spall.

Re-estimating the baseline from months that did NOT alarm cuts false alarms across the healthy years from 21 % to 6.6 % — and still catches every window of the real spall.

Lab 10 — build the alarm, not the model

Part A — see it fail

Reproduce the majority baseline, the confusion matrix and the fold-to-fold spread. Write down what each one told you that accuracy did not.

Part B — choose the operating point

Pick a cost ratio and defend it. Set the threshold from a healthy quantile. Choose m and n for the persistence rule.

Part C — write the specification

Export `lecture10_alarm.json`, including the false-alarm rate you are buying and the date the baseline expires.

Pace for the lab half

Reproduce 45 min · operating point and persistence 90 min · write the specification 45 min.

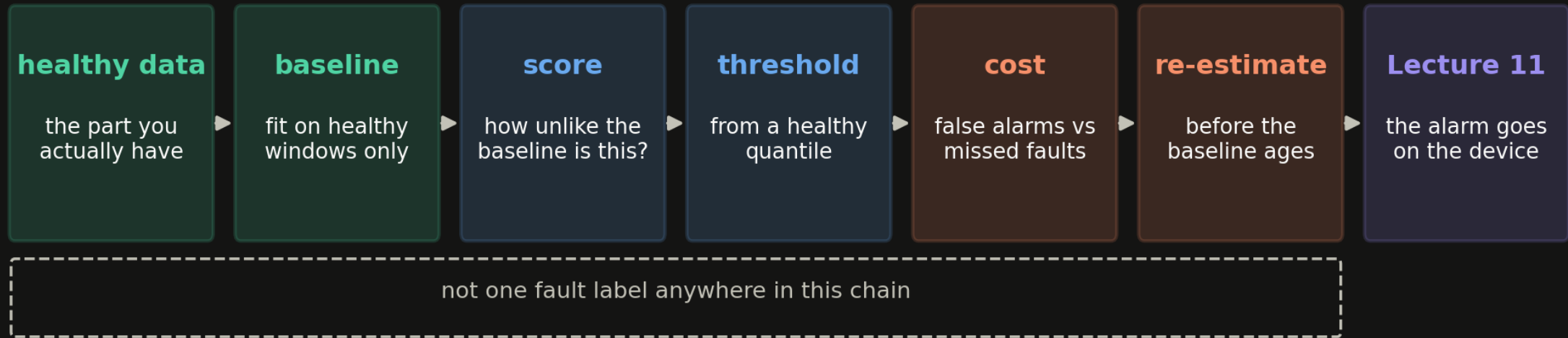
rule	FA/window	recall	healthy machines alarmed
1-of-1	0.0101	0.514	25 %
2-of-3	0.0029	0.500	6.8 %
3-of-5	0.0004	0.492	1.5 %
4-of-7	0.0000	0.475	0.0 %

The cheapest fix in the lecture

3-of-5 cuts nuisance alarms twenty-five-fold for 0.022 of recall and 256 ms of delay. No new model, no new data, no labels.

Keep `lecture10_alarm.json` — 25 numbers and a persistence rule. Lecture 11 puts it on the ESP32 alongside Lecture 9's classifier.

The score you report decides what you will build



1. Compute the majority-class accuracy first. Here it is 0.880 — from a model that never predicts a fault.
2. Report the confusion matrix. Accuracy 0.983 and bearing recall 0.577 are the same model.
3. The threshold is an engineering decision. Get the cost ratio from the people who pay the costs.
4. With 6 spalls, quote the spread: this recall is worth about ± 0.342 , not ± 0.062 .
5. Anomaly detection needs only healthy data — and matched the supervised bearing recall (0.577) with no labels at all.
6. Baselines age. A threshold set in year one fires on 48 % of a healthy machine by year four.

Generated by Claude and customized by

